

1. Why Audit after a Model Upgrade?

GPT-6 Astra follows instructions more strongly than prior models according to OpenAI's model guidance. This improves control, but it can expose a hidden tax: old instruction files and skills may have been written to compensate for weaknesses that no longer exist.

A model upgrade changes not only output quality but the optimal harness around the model.

2. Inventory

Before editing anything, list every instruction surface the task can encounter:

- global AGENTS.md or personal guidance;
- repository and nested AGENTS.md files;
- project config and profiles;
- skills and their triggers;
- plugins and connector instructions;
- subagent definitions;
- test/review automation;
- task templates or copied prompt snippets.

The inventory should identify scope and owner. Unknown provenance is itself an audit finding.

3. Classify Each Rule

For each durable instruction, ask whether it is:

Invariant: still necessary and broadly true.

Mechanically enforceable: better expressed as a test, lint rule, formatter, schema, or hook.

Workflow: should move into a skill invoked only for relevant tasks.

Historical workaround: existed because an older model could not perform a step directly.

Preference: belongs at personal/global scope rather than the repository.

Stale or contradictory: should be removed.

This taxonomy turns cleanup into a behavioral task rather than copy editing.

4. Vanilla Baseline

Kundel's Astra advice gives a particularly useful test: retry a task without extra skills first. The baseline tells you what the current model can do natively. Then add one customization at a time and measure whether it improves success, reliability, speed, or reviewability.

If the customized run is worse, do not keep the skill because it took effort to write. Sunk cost is not a workflow requirement.

5. Conflict Search

OpenAI's Astra guidance explicitly recommends looking for silent and conflicting guidance when many instruction files are loaded. Conflicts can be semantic rather than textual. "Prefer minimal edits" and "refactor adjacent code while here" do not share words, but they pull in opposite directions.

Ask the agent to enumerate active constraints before a representative task, then inspect which source introduced each. Resolve the conflict at its source rather than adding another instruction that says "really follow X."

6. Verification Rules

Keep verification guidance concrete and proportional. "Always test everything" is expensive and often ignored or over-applied. Better: identify commands and conditions that trigger them. State when visual inspection is required. State when a read-only review is expected. State when the task can stop.

Astra's tendency to keep checking work makes stopping guidance especially important.

7. Reasoning Defaults

Audit reasoning defaults too. A repository configured around high effort for an older model may spend unnecessary allowance with Astra. Run representative tasks at Low, Medium, and the existing setting. Compare corrections and completion, not aesthetics alone.

8. The Result of an Audit

The desired artifact is not a beautifully written AGENTS.md. It is a smaller, clearer instruction system in which each remaining rule has a scope, a reason, and an observable effect. Remove redundant layers. Preserve organizational knowledge. Re-run the same task suite after changes.

References: OpenAI, GPT-6 Astra model guidance; OpenAI, Codex customization; Dominik Kundel, "Five things I learned from using Astra."

9. Worked Audit

Suppose a repository has a 220-line AGENTS.md, three overlapping frontend skills, and a global instruction that says "always use the browser for UI changes." Sample ten recent tasks. You find that the global rule forces browser work even for CSS token renames, two skills duplicate the same screenshot procedure, and a legacy skill bans a component that the current design system requires. The audit should remove the obsolete ban, merge or retire duplicate workflows, and narrow browser use to changes whose acceptance criteria are visual.

10. Evaluation Matrix

For each representative task record: success without customization; success with current customization; number of user corrections; tool failures; elapsed time; usage; and whether verification evidence improved. A customization that makes outputs look more elaborate but increases corrections has not earned its place. A terse build-command rule that prevents repeated test mistakes probably has.

11. Instruction Debt

Instruction debt resembles code debt. A workaround is added under pressure, its original failure disappears, and the workaround remains because nobody can prove it is safe to remove. Over time, rules accrete into a second undocumented program controlling the agent. Version model-sensitive guidance, record why unusual rules exist, and schedule audits after major capability changes.

12. The Removal Test

For each section, temporarily remove it and run the task most likely to need it. If behavior does not materially worsen across representative cases, deletion is evidence-backed. If behavior worsens, restore the smallest rule that recovers the useful effect. This is stronger than asking whether the prose "sounds important."

13. Contradiction Budget

Every new durable rule creates another possible interaction with existing rules. Before adding one, search for instructions that already govern the same decision. Prefer editing the existing rule to stacking exceptions. The objective is a small coherent policy graph, not a museum of past failures.

Independent Open Course

<https://github.com/dicnunz/codex-astra-course>

CODEX.00 | Codex with GPT-6 Astra

Handout 21: Auditing Instructions for Astra

Fall 2026

License and source

Structural and pedagogical adaptation of MIT OpenCourseWare 24.00 Problems of Philosophy, Fall 2019, Prof. Miriam Schoenfield.

Original course and this adaptation are licensed CC BY-NC-SA 4.0.

This course is independently produced and is not affiliated with or endorsed by MIT or OpenAI.

Linked external readings and referenced product documentation retain their own copyrights and terms.

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

<https://ocw.mit.edu/courses/24-00-problems-of-philosophy-fall-2019/>